PYTHON COURSE — DAY 45 REFERENCE

JWT Authentication | 24 May 2026

Session 1 — What is JWT and Why It Matters

JWT = JSON Web Token. Sessions store user data on the server. JWT stores user data inside the token itself — the server needs no storage.

```
# JWT has 3 parts separated by dots: # eyJhbGci... . eyJ1c2Vy... . abc123xyz # HEADER PAYLOAD SIGNATURE # Header — algorithm used to sign # Payload — data (username, role, expiry) # Signature — proves token hasn't been tampered with
```

```
import jwt import datetime SECRET_KEY = 'kalikeng_secret_2026' def create_token(username, role): payload = { 'user': username, 'role': role, 'exp': datetime.datetime.now(datetime.timezone.utc) + datetime.timedelta(hours=24) } token = jwt.encode(payload, SECRET_KEY, algorithm='HS256') # singular return token def verify_token(token): try: payload = jwt.decode(token, SECRET_KEY, algorithms=['HS256']) # plural + list return payload except Exception as e: print(f'JWT Error: {e}') return None
```

Business Use: JWT is used in mobile apps, REST APIs, and microservices. When the Kalikeng tool is deployed to the cloud — multiple servers can verify the same token without sharing session storage.

Sessions vs JWT:

```
Sessions: - Server stores session data - Works well for web apps with browsers - Doesn't scale well with multiple servers JWT: - Token contains all data — server stores nothing - Works for web, mobile, APIs - Any server can verify the token - Stateless — scales perfectly
```

- pip install PyJWT — not jwt
- jwt.encode() — algorithm singular
- jwt.decode() — algorithms plural with list ['HS256']
- exp — expiry time in Unix timestamp format
- datetime.timedelta(hours=24) — token valid for 24 hours
- Token has 3 parts: Header.Payload.Signature

Session 2 — Implementing JWT in Flask

```
from functools import wraps # Decorator — protects any route with JWT def token_required(f): @wraps(f) def decorated(*args, **kwargs): token = request.headers.get('Authorization') # read from header if not token: return jsonify({'error': 'Token missing'}), 401 if token.startswith('Bearer '): token = token[7:] # remove 'Bearer ' prefix (7 chars) payload = verify_token(token) if not payload: return jsonify({'error': 'Invalid or expired token'}), 401 return f(payload, *args, **kwargs) # pass payload to route function return decorated
```

```
# Login route — returns JWT token @app.route('/api/login', methods=['POST']) def login(): data = request.get_json() username = data.get('username', '').lower() password = data.get('password', '') if username in USERS and USERS[username]['password'] == password: token = create_token(username, USERS[username]['role']) return jsonify({'token': token, 'user': username}), 200 return jsonify({'error': 'Invalid credentials'}), 401 # Protected route — requires valid JWT @app.route('/api/profile') @token_required def profile(payload): # payload passed by decorator return jsonify({ 'user': payload['user'], 'role': payload['role'] })
```

Business Use: Every Kalikeng API endpoint that handles client data, quotes, or financial information must be JWT-protected. Only authenticated users with valid tokens can access sensitive data.

- token_required decorator — applies JWT check to any route
- request.headers.get('Authorization') — reads token from request
- token[7:] — removes 'Bearer ' (7 characters including the space)
- payload passed to route function automatically by decorator
- 401 = Unauthorised — token missing or invalid
- 403 = Forbidden — authenticated but not allowed
- Always return jsonify() from API routes — never render_template

---

Session 3 — JavaScript Consuming JWT API

JavaScript does the same thing test_jwt.py did — gets a token from login, stores it, sends it with every protected request. The difference is it runs in the browser.

```
let token = ''; // stored in memory — cleared on page refresh async function login() { const username = document.getElementById('username').value; const password = document.getElementById('password').value; const response = await fetch('/api/login', { method: 'POST', headers: {'Content-Type': 'application/json'}, body: JSON.stringify({username, password}) }); const data = await response.json(); if (response.ok) { token = data.token; // store the token loadProfile(); // immediately fetch protected data } else { showMessage(data.error, 'error'); } } async function loadProfile() { const res = await fetch('/api/profile', { headers: {'Authorization': `Bearer ${token}`} // send token in header }); const data = await res.json(); document.getElementById('profile').textContent = `User: ${data.user} | Role: ${data.role}`; }
```

Business Use: The taxi management tool login page — driver or owner enters credentials, JavaScript gets the JWT token, all subsequent API calls include the token. Owner sees all data, driver sees only their own trips.

- let token = '' — declare outside functions so all functions can access it
- Token stored in JS variable — lost when page refreshes
- Authorization: Bearer [token] — standard HTTP header format
- Browser address bar can't send custom headers — must use JavaScript fetch
- response.ok — true if status 200-299
- loadProfile() called immediately after successful login
- type='button' on login button — prevents form default submit behaviour